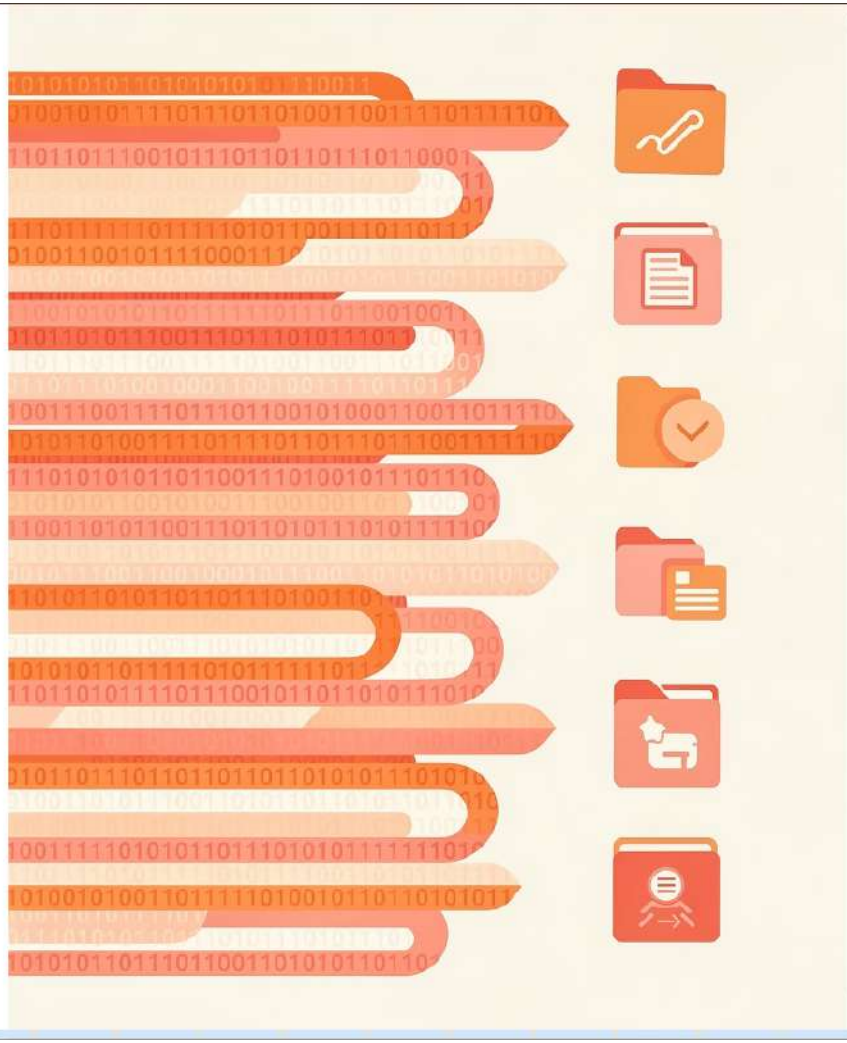
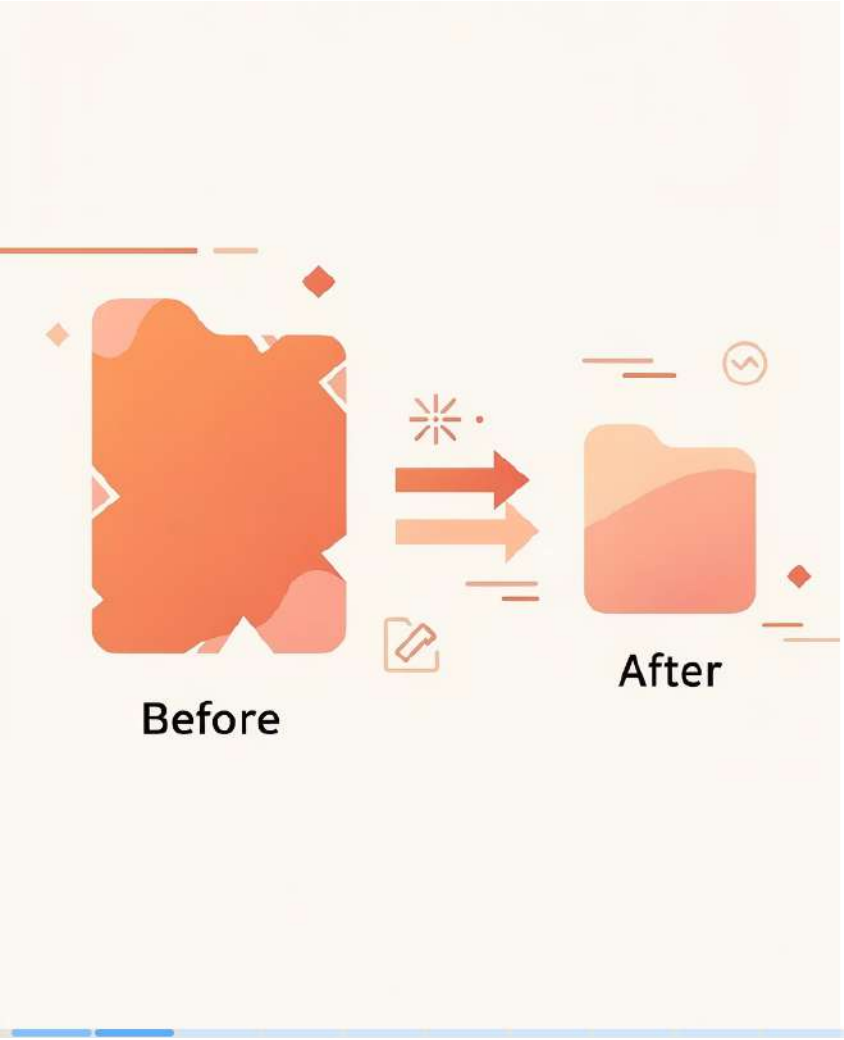
Compression and Decompression Algorithms

Understanding how digital files shrink and expand across common formats





What Is Compression?

Compression

The process of reducing data size by eliminating redundancy, irrelevance, or unnecessary information. Essential for efficient storage, faster transmission over networks, and reduced bandwidth usage.

Decompression

The reverse process of reconstructing the original or near-original data from the compressed form. Restores files to usable formats.

Two Types of Compression

Lossless Compression

Original data can be perfectly reconstructed after decompression. No information is lost in the process.

- Perfect reconstruction
- Lower compression ratios
- Used for text and archives

Lossy Compression

Some data is permanently removed to achieve higher compression ratios, with acceptable quality loss.

- Higher compression ratios
- Irreversible data loss
- Used for media files

These techniques are widely applied in **images, audio, video, and document file formats**.

Four Types of Redundancy

1

Spatial Redundancy

Repetition of similar data, such as neighboring pixels in images that share the same color.

2

Temporal Redundancy

Similarity between consecutive frames in videos where most content remains unchanged.

3

Statistical Redundancy

Unequal probability of symbols appearing in data, allowing frequent symbols to use shorter codes.

4

Perceptual Redundancy

Information less noticeable to human senses that can be removed without significant quality loss.

Core Encoding Techniques

Entropy Encoding

Uses statistical properties to assign shorter codes to frequent symbols. Examples: Huffman coding assigns variable-length codes, while Arithmetic coding represents entire messages as single numbers.

Dictionary-Based

Builds a dictionary of repeated patterns and replaces them with references. LZ77, LZ78, and LZW algorithms identify and encode repeating sequences efficiently.

Transform Coding

Converts data into frequency domain where it can be compressed more effectively. DCT and DWT separate important from less important information.

Predictive Coding

Predicts future values based on past data and encodes only the difference between prediction and actual value.

DEFLATE: The Power of Two

DEFLATE is a popular lossless data compression algorithm that combines two powerful techniques to achieve high compression ratios:

LZ77 Algorithm

This dictionary-based method identifies and replaces repeating sequences of data with short references (back-pointers). It effectively reduces spatial and temporal redundancies within the data stream.

Huffman Coding

An entropy encoding technique that assigns variable-length codes to the data generated by LZ77. More frequent symbols receive shorter codes, optimizing for statistical redundancy.

By leveraging the strengths of both LZ77 and Huffman coding, DEFLATE achieves highly efficient lossless compression, making it a cornerstone for formats like ZIP, PNG, and GZIP.

LZ77 and Huffman in Action

Let's look at simplified examples of how LZ77 and Huffman coding contribute to efficient data compression.

LZ77: Replacing Repetition

LZ77 identifies and replaces repeating sequences of data with short references. These references typically point back to an earlier occurrence of the same sequence.

Example: Consider the string `ABCAABCAABC`

- First `ABCA` is output.
- The next `ABC` is replaced by a pointer: `(4,3)` (offset 4 characters back, length 3 characters).
- The final `ABC` is also replaced by a pointer: `(7,3)`.

Result: `ABCA(4,3)(7,3)`

Huffman: Shortening Frequent Symbols

Huffman coding assigns variable-length codes to data based on frequency. More frequent symbols receive shorter codes, while less frequent ones get longer codes, minimizing overall data size.

Example: Encoding the string `AAABBC`

- `A` (3 occurrences): Assign `0`
- `B` (2 occurrences): Assign `10`
- `C` (1 occurrence): Assign `11`

Result: `000101011` (9 bits, compared to 18 bits for fixed-length 3-bit codes)

PNG: Lossless Image Compression

How PNG Works

PNG uses the **DEFLATE algorithm**, combining LZ77 and Huffman coding for efficient lossless compression.

Compression Process:

1. Image data is filtered to reduce spatial redundancy
2. LZ77 identifies repeating byte sequences
3. Huffman coding compresses symbols based on frequency

Decompression: Reverse DEFLATE process followed by inverse filtering restores the original image perfectly.

Best for: Logos, screenshots, images with sharp edges and transparency

JPEG: Lossy Image Compression

01

Color Space Conversion

RGB → YCbCr separates brightness from color information

02

Chrominance Subsampling

Reduces color data while preserving brightness detail

03

Block Splitting

Divides image into 8×8 pixel blocks for processing

04

DCT Transform

Converts spatial data to frequency domain

05

Quantization

Lossy stage that discards less important frequencies

06

Entropy Encoding

Huffman or Arithmetic coding compresses remaining data

Decompression: Inverse quantization → IDCT → color space conversion. Perfect for photographs and web images.

Modern Image Formats

GIF - Graphics Interchange Format

Algorithm: LZW (Lempel-Ziv-Welch)

Builds a dictionary of repeated pixel patterns for compression. Reconstructs image data using the same dictionary during decompression.

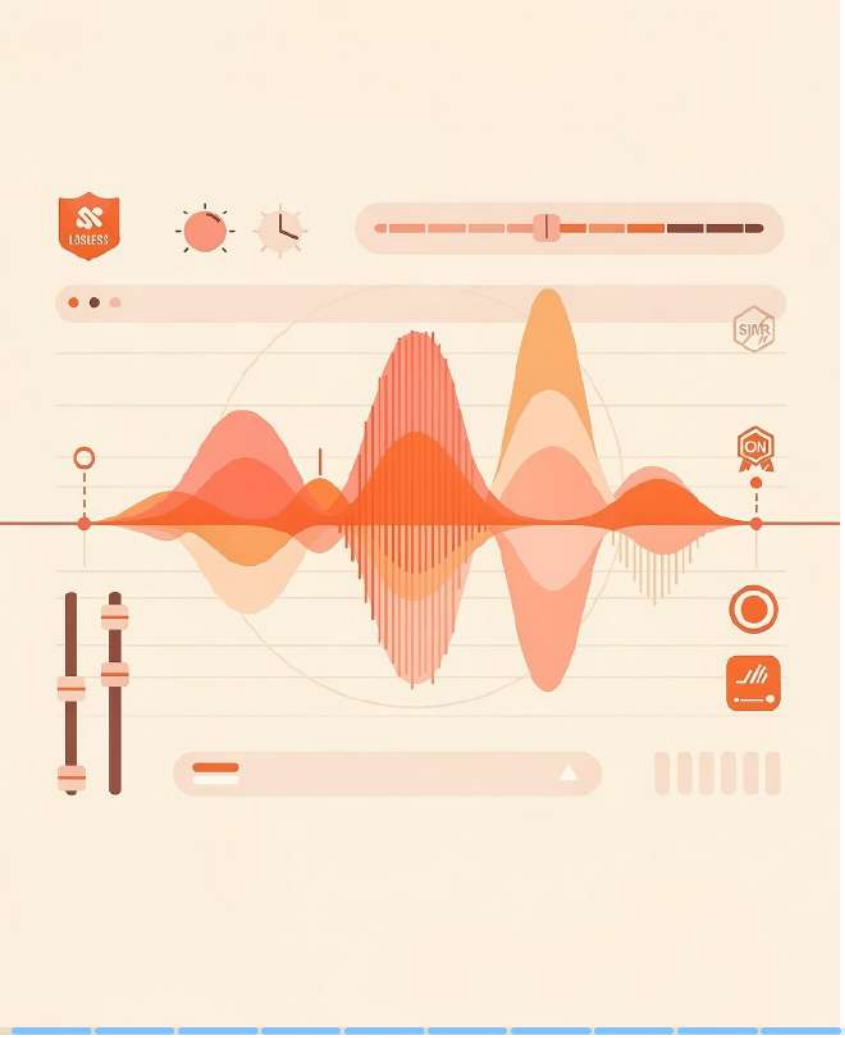
Limitation: 256-color palette restricts use to simple graphics and animations.

WebP - Modern Web Format

Lossy: VP8 intra-frame prediction + DCT-like transforms

Lossless: Advanced LZ77 with entropy coding

Advantage: Achieves smaller file sizes than JPEG and PNG at similar quality levels, ideal for web optimization.



FLAC: Lossless Audio Compression

How FLAC Works

FLAC (Free Lossless Audio Codec) uses **linear prediction combined with Rice coding** to compress audio without any quality loss.

Compression Process

- Predicts audio samples using linear predictors
- Calculates the residual error (difference between prediction and actual)
- Encodes the residual error efficiently using Rice coding

Decompression

Reconstructs original samples exactly by reversing the prediction process. No information is lost.

Use cases: Archival storage and high-quality music collections

MP3: Lossy Audio Compression

Frequency Transform

MDCT converts audio to frequency domain

Psychoacoustic Analysis

Removes sounds humans can't hear

Quantization

Reduces precision of remaining data

Huffman Encoding

Final compression of audio data

Decompression: Inverse MDCT reconstructs the audio signal. Widely used for music streaming and portable audio devices.

AAC (Advanced Audio Coding) offers improved psychoacoustic modeling over MP3, delivering better quality at lower bitrates.

Video Compression Fundamentals

Intra-frame Compression

Compresses individual frames similar to image compression techniques.

Inter-frame Compression

Uses differences between consecutive frames to reduce redundancy.

Motion Compensation

Tracks movement between frames to predict and encode changes efficiently.

H.264 / AVC: Industry Standard

Advanced Video Coding

H.264 delivers high compression efficiency through sophisticated algorithms, making it the standard for streaming, Blu-ray, and video conferencing.

Intra/Inter Prediction

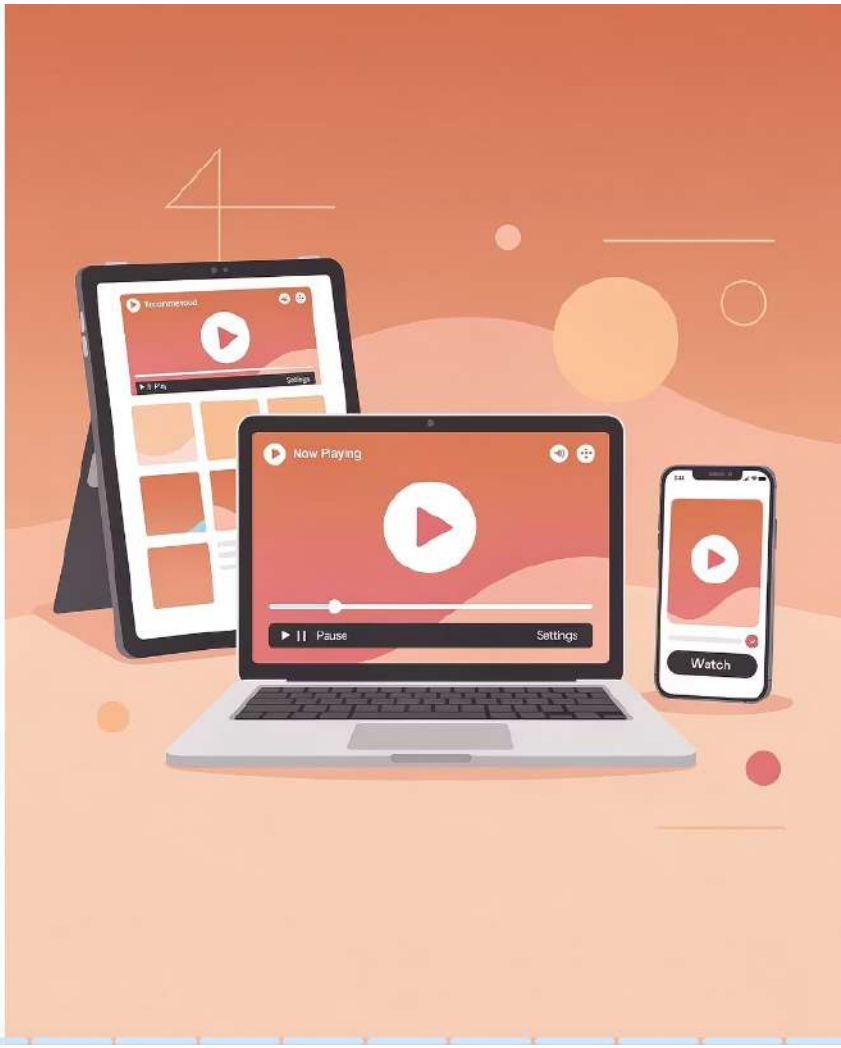
Predicts blocks from neighboring or previous frames

Integer DCT

Transform coding optimized for integer math

CABAC/CAVLC

Context-adaptive entropy coding for maximum efficiency



Evolution of Video Codecs

1. **MPEG-2**

   Used in DVDs and digital TV. Uses DCT, motion compensation, and entropy coding.

2. **H.264 / AVC**

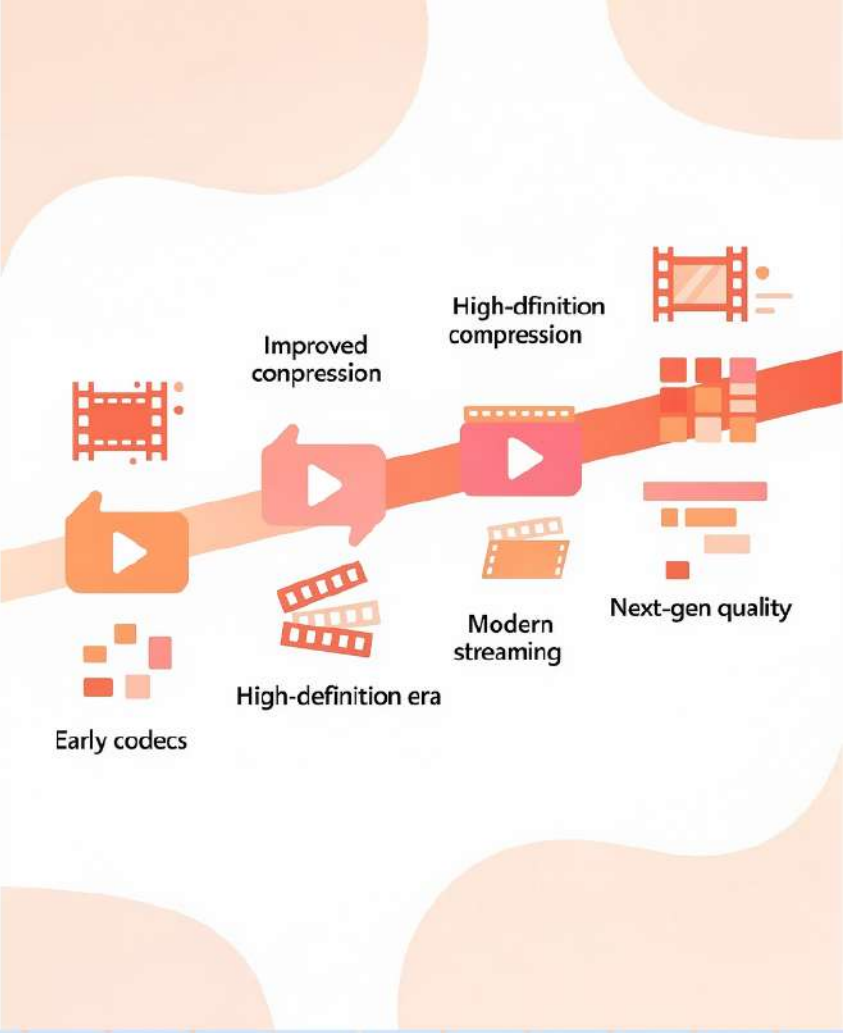
   High compression efficiency. Standard for streaming and Blu-ray.

3. **H.265 / HEVC**

   Uses larger block structures (CTUs). ~50% bitrate reduction vs H.264.

4. **VP9 and AV1**

   Open and royalty-free codecs. Used by YouTube, Netflix, and web platforms.



ZIP: Universal Document Compression

The DEFLATE Algorithm

ZIP files use the **DEFLATE algorithm**, combining two powerful compression techniques:

Compression Process:

1. **LZ77 dictionary compression** identifies and replaces repeating patterns with references
2. **Huffman encoding** assigns shorter codes to frequently occurring symbols

Decompression: Reverses the DEFLATE process to perfectly reconstruct original files.

RAR format uses advanced dictionary compression and prediction for higher compression ratios than ZIP.

PDF: Multi-Method Compression

PDF (Portable Document Format) is designed for reliable document exchange, and its robust compression strategy is key to achieving this. Unlike simpler formats, PDF employs a sophisticated, multi-method approach, leveraging different algorithms for various content types to optimize both file size and visual fidelity.

Historical Context: Adobe's Vision for Portable Documents

When Adobe introduced PDF in 1993, the primary goal was to create a truly "portable document" that would look and print identically on any device, regardless of the operating system, hardware, or installed fonts. This vision necessitated effective compression to ensure documents remained compact enough for easy sharing over nascent internet connections and storage on limited disk space, without compromising the integrity or appearance of the original content.

Multi-Layered Compression Architecture

PDF compression operates on several levels, allowing for granular control and efficient handling of diverse elements within a document:

01

Object-Level Compression

Individual PDF objects (like fonts, images, text blocks, vector graphics, etc.) can be compressed independently. This modularity allows for applying the most appropriate compression algorithm to each specific type of content.

02

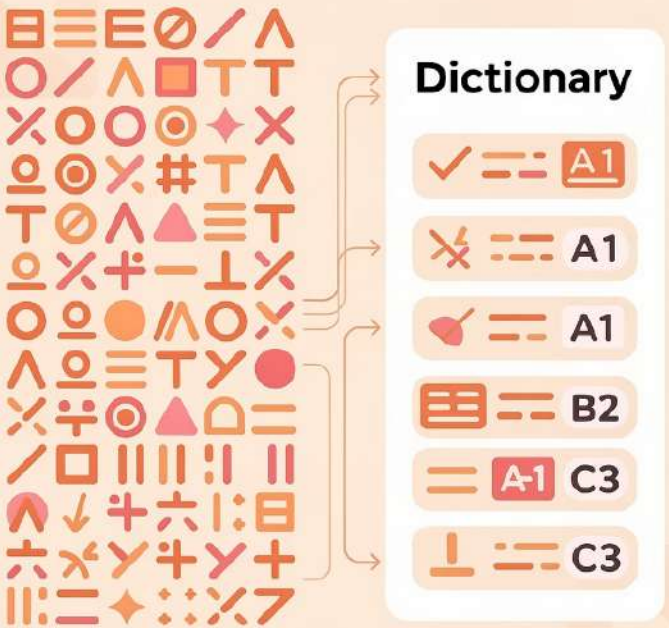
Stream-Level Compression

The actual content streams, which define the visual appearance and content of a page, are often compressed. These streams contain instructions for rendering text, drawing graphics, and placing images.

03

File-Level Optimization

Beyond individual objects, PDFs can undergo further optimization, such as linearization (for web-optimized viewing) or object stream compression (combining multiple objects into a single compressed stream) to reduce overall file size and improve parsing efficiency.



Flate/DEFLATE for Text and Vector Graphics

Flate compression, based on the widely used DEFLATE algorithm (also found in ZIP files), is the primary method for text content and vector graphics in PDFs. It is a lossless compression technique that combines two powerful methods:

LZ77 Dictionary Compression

Identifies and replaces repeating sequences of bytes (like common words, font definitions, or drawing commands) with short references to previously occurring data. Text, especially, has high redundancy due to repeated characters, words, and structural elements (e.g., in programming code or formatted documents).

Huffman Coding

Assigns variable-length codes to the output of LZ77. More frequently occurring symbols or references receive shorter codes, while less frequent ones get longer codes, further reducing the overall data size.

Why Perfect for Text: Flate compression is ideal for text because of the inherent redundancy in language and font data, ensuring perfect reconstruction without any loss of quality.

JPEG and JPEG2000 for Images

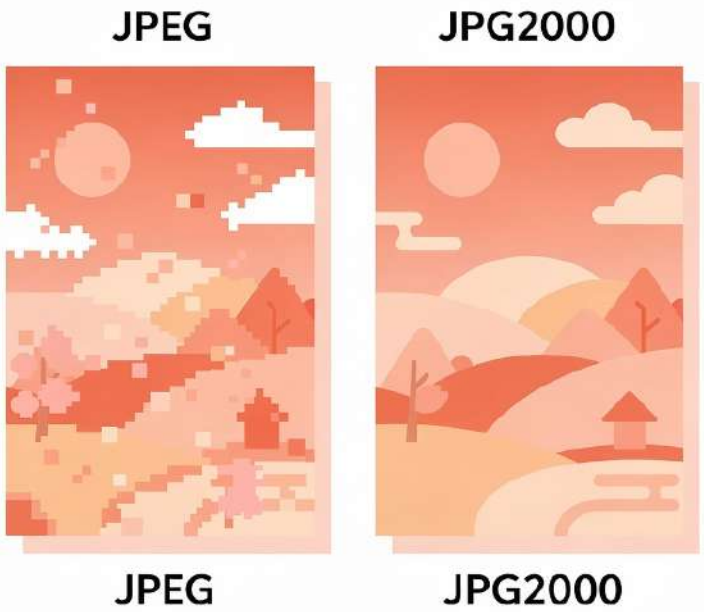
PDF uses different compression methods for photographic and continuous-tone images:

JPEG (DCTDecode) - Standard Image Compression

- **Discrete Cosine Transform (DCT)**: Transforms image data from the spatial domain to the frequency domain, separating high-frequency (detail) and low-frequency (general color) components.
- **Quantization**: Reduces the precision of frequency coefficients, discarding less important data. This is where the lossy compression occurs; higher compression ratios mean more aggressive quantization and thus more quality loss.
- **Quality Settings**: Users can choose quality settings (e.g., 80% quality) which directly affect the quantization table and final file size.

JPEG2000 (JPXDecode) - Advanced Compression

- **Wavelet-Based Compression**: Uses wavelet transforms instead of DCT, which is more effective at representing images at different resolutions and typically produces better visual quality at high compression ratios compared to standard JPEG.
- **Progressive Decoding**: Allows images to be decoded and displayed gradually, from a low-resolution blurry version to a high-resolution sharp one, which is beneficial for web viewing.



JBIG2 and CCITT for Monochrome Images

For black and white images like scanned documents, PDF uses specialized compression methods:

JBIG2 - Pattern Matching for Scanned Documents

JBIG2 (Joint Bi-level Image Experts Group 2) is specifically designed for highly efficient compression of bi-level (black and white) images, such as scanned documents, faxes, or line art.

- **Pattern Matching and Substitution**: Identifies repeating patterns (e.g., individual characters or glyphs) within the image. Each unique pattern is stored once in a shared dictionary, and subsequent occurrences are replaced with a compact reference to that dictionary entry.
- **Extreme Compression**: This is incredibly effective for text-heavy scanned documents where the same characters appear repeatedly. Can achieve compression ratios of 100:1 or more.
- **Arithmetic Coding**: Used to further compress the pattern data and references.

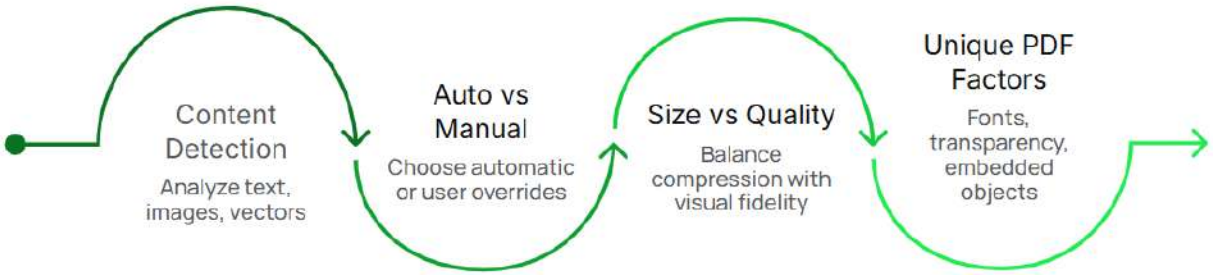
CCITT Group 3 and Group 4 - Fax-Style Compression

These are legacy compression methods primarily used for bi-level (black and white) images, originating from facsimile transmission standards. They are lossless.

- **Run-Length Encoding**: Both methods use variations of run-length encoding, which compresses data by storing the number of consecutive pixels of the same color.
- **Group 4 Advantage**: Offers better compression by comparing scan lines.

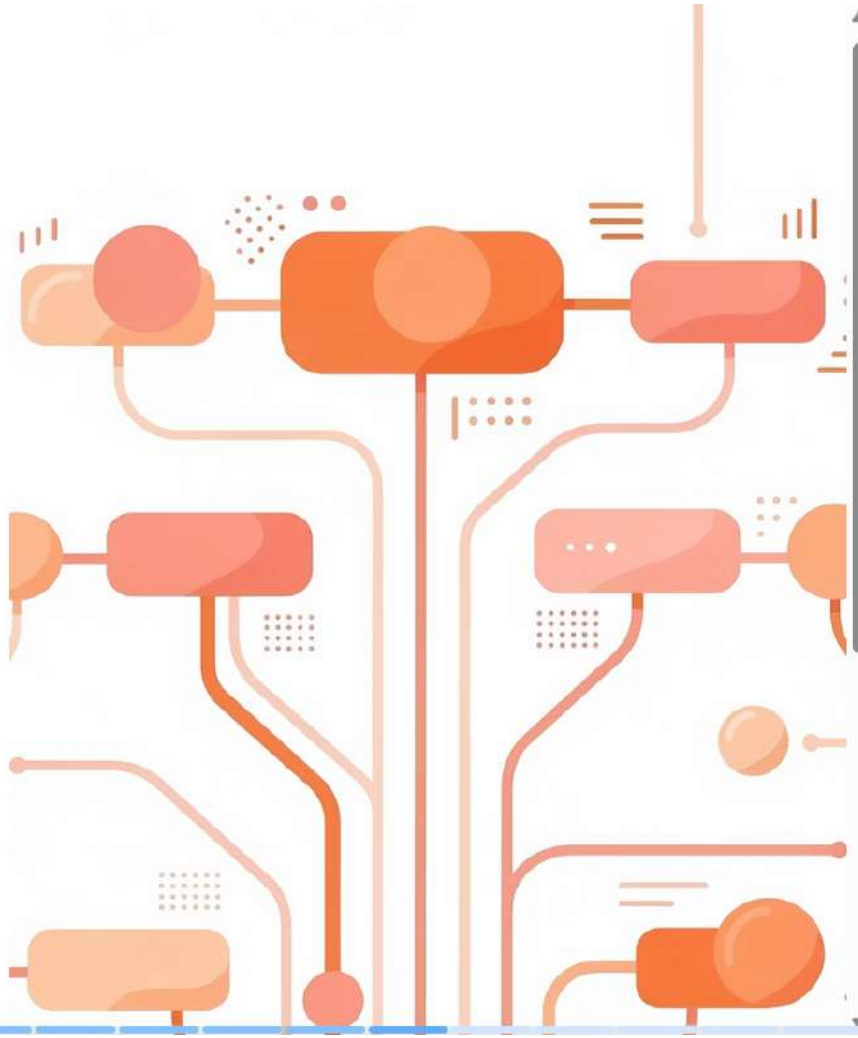
How PDF Decides Which Compression to Use

PDF generators and editors employ sophisticated logic to determine the optimal compression strategy:



Content Type Detection
Software analyzes the nature of each object within the document (e.g., recognizing text, distinguishing photographs from line art, identifying vector graphics).

Automatic vs. Manual Settings
Most PDF creation tools offer automatic settings that apply suitable compression based on content, often with predefined "Standard," "Print," or "Web" optimization profiles. Users can also manually override these to specify desired quality levels for images (e.g., "High," "Medium," "Low" JPEG quality) or choose specific algorithms.



Content Type Detection
Software analyzes the nature of each object within the document (e.g., recognizing text, distinguishing photographs from line art, identifying vector graphics).

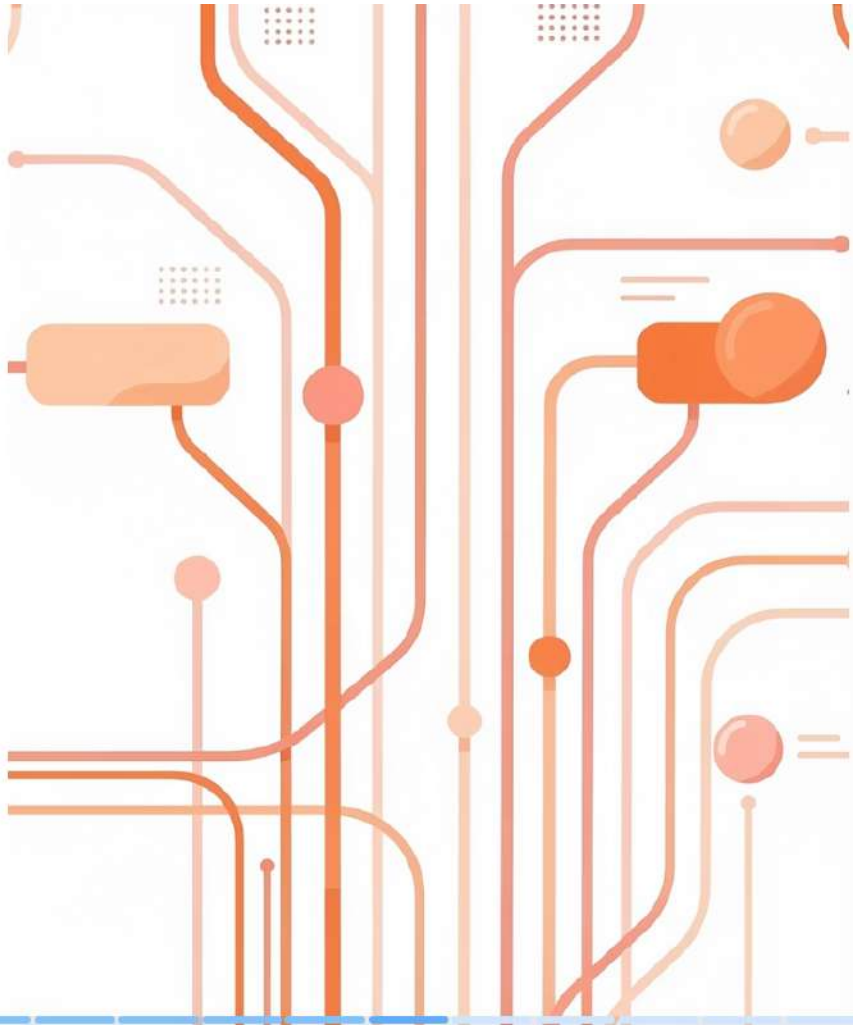
Automatic vs. Manual Settings
Most PDF creation tools offer automatic settings that apply suitable compression based on content, often with predefined "Standard," "Print," or "Web" optimization profiles. Users can also manually override these to specify desired quality levels for images (e.g., "High," "Medium," "Low" JPEG quality) or choose specific algorithms.

Balancing File Size vs. Quality
The decision-making process often involves a trade-off:

- **Lossy compression methods** (JPEG, JBIG2) can drastically reduce file size but introduce some data loss
- **Lossless methods** (Flate, CCITT, LZW) guarantee perfect reconstruction but might yield larger files
- The choice depends on the intended use of the PDF (e.g., high-quality print vs. fast web download)

PDF's Unique Features:

- **Incremental Updates**: Only changes are appended and compressed, not the entire document
- **Font Subsetting**: Embeds only necessary characters, reducing file size
- **Metadata Preservation**: Maintains document structure and searchability
- **Cross-Reference Table Compression**: Optimizes file structure for large documents



Real-World Example: Comprehensive PDF Document

Imagine a typical PDF report containing multiple content types. Here's how PDF applies different compression methods simultaneously:

Title Page and Body Text

Method: Flate (DEFLATE) compression

Why: Leverages redundancy in characters, words, and formatting

Result: Perfect reconstruction with significant size reduction

Company Logo (Vector Graphic)

Method: Flate (DEFLATE) compression

Why: Lossless compression of drawing commands and paths

Company Logo (Vector Graphic)

Method: Flate (DEFLATE) compression

Why: Lossless compression of drawing commands and paths

Result: Scalable graphic with minimal file size

Product Photograph

Method: JPEG or JPEG2000 compression

Why: Controlled balance between visual quality and file size

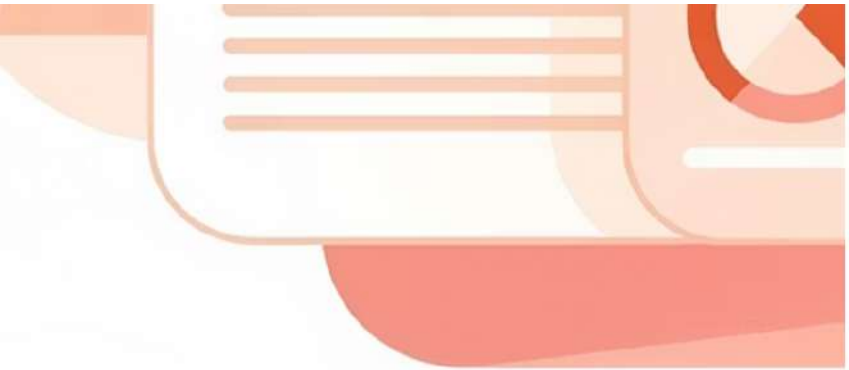
Result: High-quality image at 70-80% of original size

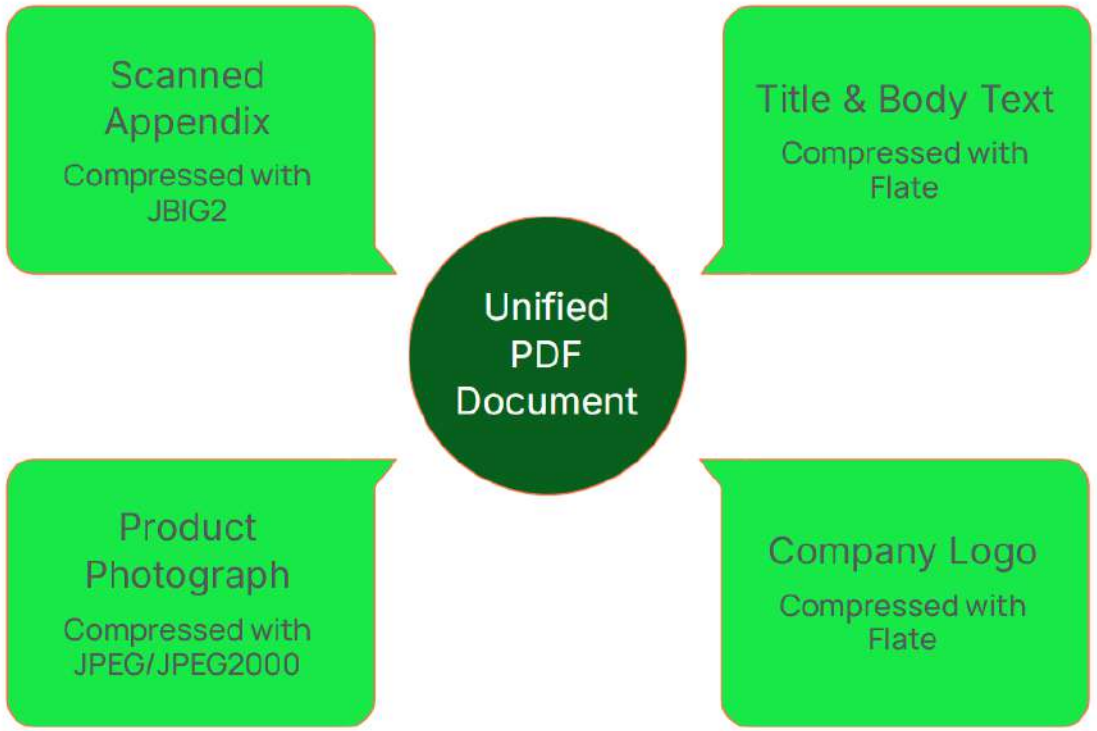
Scanned Appendix (Black & White)

Method: JBIG2 compression

Why: Pattern recognition for repeated characters

Result: Extreme compression (100:1 ratio) while maintaining readability





The Result: Each element exists as a separate object within the PDF, receiving its optimal compression method simultaneously. This creates a compact, portable, and visually accurate document that maintains quality where needed while minimizing file size.

Microsoft Office Formats

ZIP-Based Document Compression

Modern Microsoft Office formats (DOCX, PPTX, XLSX) are based on ZIP compression technology.

Structure

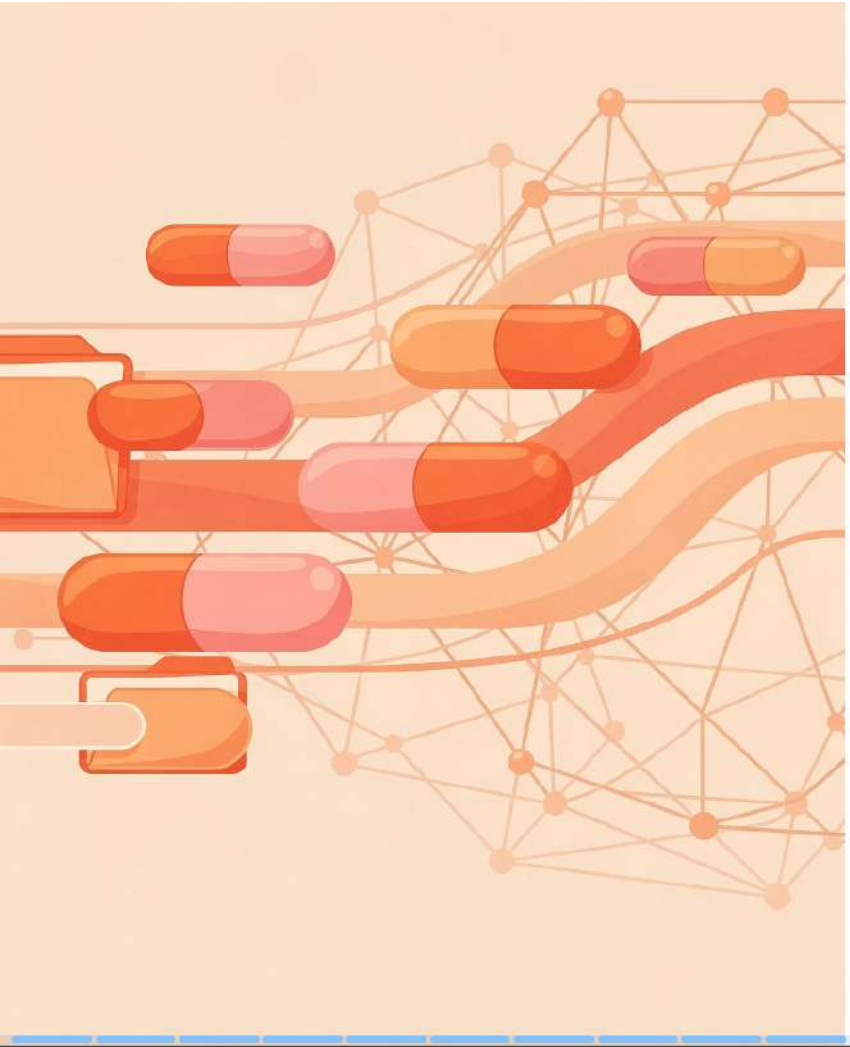
- Documents stored as XML files
- Images and media embedded separately
- All content packaged in ZIP container

Compression

- XML files compressed using DEFLATE
- Significant size reduction vs legacy formats
- Easy extraction and modification

Compression Comparison Table

Media Type	Format	Type	Algorithm
Image	PNG	Lossless	DEFLATE
Image	JPEG	Lossy	DCT + Huffman
Audio	MP3	Lossy	MDCT + Psychoacoustic
Audio	FLAC	Lossless	Linear Prediction
Video	H.264	Lossy	Motion Compensation + DCT
Document	ZIP	Lossless	DEFLATE



Advantages of Compression

Reduced Storage Requirements

Compression dramatically decreases the space needed to store files, allowing more data on the same hardware.

Faster Data Transmission

Smaller file sizes mean quicker uploads, downloads, and transfers across networks and the internet.

Efficient Multimedia Streaming

Enables real-time delivery of high-quality video and audio content over limited bandwidth connections.

Limitations and Challenges

Irreversible Data Loss

Lossy compression permanently removes information. Once compressed, the original quality cannot be fully restored, making it unsuitable for archival purposes.

Computational Complexity

Advanced codecs like H.265 and AV1 require significant processing power for encoding and decoding, impacting battery life and performance on mobile devices.

Compatibility Issues

Different formats and codecs may not be universally supported across all devices and platforms. Licensing fees for proprietary codecs can also limit adoption.